Aleph 4 Agents

«il luogo dove sono, senza confondersi, tutti i luoghi [...] visti da tutti gli angoli»
— Jorge Luis Borges, *L'Aleph*

Un linguaggio di programmazione per agenti. Specifica, architettura, implementazione e adozione — documento v1.0, 10 novembre 2026.

Indice

1. Visione, obiettivi e non-obiettivi · 2. Principi di design · 3. Stato dell'arte · 4. Specifica del linguaggio · 5. Architettura del toolchain · 6. Metodologia sperimentale · 7. Piano di implementazione · 8. Strategia di adozione · 9. Altre idee · 10. Rischi · 11. Decisioni aperte · 12. Riferimenti

1. Visione, obiettivi e non-obiettivi

Costruire un linguaggio di programmazione il cui **autore e lettore primario sono agenti AI**. I linguaggi attuali sono stati progettati per cervelli umani: sintassi esprimibile, indentazione visiva, nomi lunghi, messaggi di errore in prosa. Un agente invece produce token in sequenza, consuma una finestra di contesto finita e paga ogni token, ogni retry e ogni lettura. Il linguaggio va progettato su questa realtà.

Visione di prodotto: le persone parlano al computer a voce; sotto, agenti scrivono, verificano ed eseguono programmi in un linguaggio compatto, veloce e sicuro, che nessuno ha bisogno di leggere — ma che resta **ispezionabile** quando serve.

Perché Aleph 4 Agents: nel racconto di Borges l'Aleph è un punto che contiene tutto l'universo senza confusione. È l'idea del progetto: un'unica forma, compatta, che rappresenta qualsiasi programma senza ambiguità, leggibile da qualsiasi agente. È anche la prima lettera dell'alfabeto ebraico: il linguaggio di partenza. Nel testo il linguaggio si chiama semplicemente Aleph; il nome completo **Aleph 4 Agents** lo distingue da altri progetti storici con lo stesso nome.

Obiettivi

- **Pochi token**: meno token per programma rispetto a Python/TypeScript/Rust, misurati sui tokenizer reali.
- **Velocità nativa**: compilazione AOT a codice nativo, senza interprete e senza garbage collector nel percorso normale.
- **Correttezza al primo colpo**: sintassi e tipi che rendono difficile scrivere codice sbagliato; ciclo di correzione rapido e strutturato.
- **Sicurezza per default**: nessun comportamento indefinito, permessi dichiarati ed imposti, esecuzione in sandbox possibile.
- **Apprendibilità in-context**: un modello che non l'ha mai visto deve poterlo scrivere bene leggendo una spec breve.

Non-obiettivi (per la v1)

- Non sostituire i linguaggi umani per tutto: partiamo da una nicchia (vedi sezione 8).
- Non essere "elegante" o leggibile a colpo d'occhio: la leggibilità umana passa dalla vista umana generata, non dal sorgente.
- Non avere un ecosistema di librerie ampio: libreria standard piccola ma completa sulle operazioni che gli agenti usano di più, più FFI verso C.

- Non supportare metaprogrammazione, macro o ereditarietà.

2. Principi di design

#	Principio	Motivazione
1	**Token-first, misurato**	Ogni parola chiave, simbolo e costrutto si sceglie sul tokenizer reale, non a intuito. Ogni modifica si valida con il benchmark.
2	**Una sola forma**	Per ogni programma esiste un solo testo valido. Niente varianti di stile: meno scelte per il modello, meno diff inutili, caching migliore.
3	**Pochi passi validi**	A ogni token il numero di continuazioni valide dev'essere piccolo. Meno scelte, meno errori, meno retry.
4	**Errori impossibili o precoci**	Quello che si può rendere non rappresentabile (grammatica, tipi) non va lasciato al runtime.
5	**AST come verità, testo come vista**	Il programma è un albero con ID stabili. Il testo compatto è una proiezione; ne esistono altre (JSON, vista in italiano).
6	**Nessun comportamento nascosto**	Niente eccezioni implicite, conversioni automatiche, overload, macro. Il percorso di esecuzione si legge dal codice.
7	**Sicuro per default**	Capacità dichiarate, sandbox, niente undefined behavior. Il codice non letto da nessuno non può avere poteri impliciti.
8	**Feedback per macchine**	Ogni strumento restituisce dati strutturati, non prosa. Il ciclo scrivi-controlla-correggi deve durare meno di un secondo.
9	**Il vocabolario non è troppo piccolo**	Un vocabolario minimo costringe gli agenti a reinventare primitive. Le operazioni comuni (JSON, HTTP, stringhe, liste) sono built-in.

3. Stato dell'arte

Il campo è affollato ma immaturo: tutti progetti sperimentali, spesso individuali, nessuno con adozione industriale. Va studiato prima di scrivere codice, per rubare le idee buone ed evitare quelle già scartate.

Progetto	Idee rilevanti	Cosa prendere / evitare
ilo	Notazione prefissa, token-minimal, manifesto con principi, osservazioni empiriche su nomi e tokenizer.	Prendere: metodo di misura. Notare: alias opachi riducono l'accuratezza; spec-in-context ha un costo.
NanoLang	Transpila a C, test obbligatori, sintassi non ambigua.	Prendere: test obbligatori, backend C.
NERD	Compila a LLVM IR senza runtime; llms.txt per insegnare il linguaggio.	Prendere: llms.txt, assenza di runtime.
Ax	Sorgente compatto come sintassi canonica, binari nativi, runtime in C.	Prendere: forma canonica non-minificata.
Unison	Codice indirizzato per contenuto: ogni definizione è identificata dall'hash del suo AST.	Prendere: hash come identità, niente conflitti di dipendenze, riuso di funzioni già verificate.
Tacit	Forma canonica indipendente da nomi e ordine, nessuna indentazione.	Prendere: canonicalizzazione; valutare rischio nomi opachi.
ll-lang	Linguaggio funzionale tipato, diagnostica strutturata, transpila verso F#/TS/Python/Java/C#.	Prendere: ciclo scrivi-controlla con diagnostica; target multipli.
NXL, KARN, NURL, GlyphLang	Riduzioni di token con simboli, primitive per agenti, backend LLVM.	Prendere: dati di benchmark; evitare simboli che si spezzano in più token.

In letteratura esiste anche il lavoro su "AI-oriented grammar", che misura la riduzione di token ottenibile cambiando la grammatica su centinaia di migliaia di file Python e 14 tokenizer. Conferma che il risultato dipende dal tokenizer: quindi si misura sempre sui tokenizer dei modelli che ci interessano. Link in sezione 12.

Azione fase 0: eseguire gli stessi 5 task sui progetti esistenti, con gli stessi modelli e le stesse metriche, prima di decidere se costruire o contribuire.

4. Specifica del linguaggio

Nota: gli esempi di sintassi sono **illustrativi, non definitivi**. La sintassi finale si sceglie con gli esperimenti della sezione 6.

4.1 Lessico e scelta dei token sul tokenizer

È la caratteristica più distintiva: il vocabolario del linguaggio si **ottimizza contro i tokenizer** dei modelli target. Lo stesso testo costa un numero diverso di token a seconda del modello, e lo stesso lessema può tokenizzare in modo diverso a seconda del contesto (con o senza spazio davanti, dopo un simbolo, a inizio riga).

Procedura di selezione del lessico

1. **Candidati**: per ogni concetto (dichiarare funzione, condizione, ciclo, ritorno, tipi base, operatori) si raccolgono parole inglesi comuni e simboli ASCII.
2. **Misura**: ogni candidato si tokenizza con i tokenizer target in contesti reali (inizio riga, dopo spazio, dopo simbolo, dentro sequenze). Per i modelli senza tokenizer pubblico si usa l'API di conteggio token del fornitore.
3. **Punteggio**: preferire lessemi di 1 token in tutti i contesti e su tutti i tokenizer; penalizzare quelli che si spezzano, quelli instabili tra modelli e quelli ambigui per via di priori forti da altri linguaggi.
4. **Da evitare**: parole composte con trattino (il trattino forza una divisione), sequenze rare di simboli, Unicode esotico, numeri lunghi, indentazione variabile (i token di spazio sono costo puro).
5. **Validazione generativa**: il conteggio non basta. Si fa scrivere al modello i task del benchmark con il lessico candidato e si misura la correttezza. Un lessico a 1 token ma confuso con altri identificatori può perdere accuratezza.
6. **Versionamento**: il lessico è un **file dati** (es. lexicon.toml) separato dalla grammatica. Si ripete l'ottimizzazione quando escono nuovi tokenizer.

Conseguenza architetturale importante: poiché il testo canonico è la stampa dell'AST (4.2), **cambiare lessico = ristampare l'AST**. Le migrazioni tra edizioni del linguaggio sono automatiche e senza perdita. Si possono anche definire **profili di lessico per famiglia di modelli** per uso in-context; per training e fine-tuning va però usato un solo lessico standard, altrimenti il corpus si frammenta.

4.2 Forma canonica unica

- **Relazione 1-a-1** tra testo e AST: il parser rifiuta qualsiasi testo che non sia la stampa canonica del proprio AST (modalità strict, con messaggio che indica la forma corretta).
- **Layout deterministico**: spazi, a capo e ordine delle dichiarazioni sono fissati. Dichiarazioni ordinate in modo stabile (per hash o nome) così un file ha una sola forma.
- **Nessuna indentazione significativa**: i blocchi sono delimitati; l'indentazione, se esiste, è solo di stampa.

- **Commenti**: non esistono come testo libero; esistono come **annotazioni strutturate** sull'AST (intento, esempi, note), che non influenzano l'esecuzione e si possono omettere nella vista compatta.
- **Nomi**: i simboli pubblici hanno nomi di una parola scelti dal lessico; i nomi locali sono liberi ma di una parola. La normalizzazione dei locali (es. per posizione) è un'opzione da testare: l'esperienza di ilo suggerisce che nomi troppo opachi peggiorano la generazione. Vedi decisioni aperte.
- **Benefici**: caching dei prefissi migliore, diff minimi, meno token sprecati su stile, meno varianti da imparare.

4.3 Grammatica

- **Deterministica e a lookahead minimo** (LL(1) o quasi): ogni token successivo determina la produzione senza backtracking. Parser a discesa ricorsiva scritto a mano.
- **Pubblicata in formati per la decodifica vincolata**: EBNF di riferimento, più export in GBNF (llama.cpp) e in formati usati da motori di constrained decoding come XGrammar, llguidance, Outlines. Con un'API che permette al server di inferenza di sapere quali token sono validi a ogni passo.
- **Decodifica vincolata dai tipi** (fase successiva): oltre alla sintassi, il compilatore espone quali identificatori e tipi sono ammissibili in una posizione, restringendo ulteriormente i token candidati. Linea di ricerca recente, da valutare.
- **Due varianti sintattiche da confrontare**: (A) notazione prefissa ad arità fissa, senza parentesi (massimo risparmio di token, ma un errore di arità si propaga); (B) forma delimitata, stile S-expression, un po' più verbosa ma più robusta. La scelta si fa con A/B sul benchmark.
- **Operatori**: nessun overload, precedenza non necessaria (con prefissa o delimitata non esiste ambiguità).

Esempio illustrativo (variante A):

```
fn fib n:i64 -> i64
  ex fib 10 = 55
  ex fib 0 = 0
  if < n 2 n + fib - n 1 fib - n 2
```

L'esempio mostra la funzione, due esempi verificati dal compilatore (`ex`) e il corpo senza parentesi. La leggibilità umana è bassa per scelta; la vista umana la traduce.

4.4 Sistema dei tipi

- **Tipizzazione statica** con inferenza locale; firme di funzione sempre esplicite (aiutano modello e compilatore).
- **Tipi algebrici** (somme e prodotti), **Option** e **Result** al posto di null ed eccezioni.
- **Nessuna conversione implicita**, nessun subtyping, nessun overload.
- **Tipi etichettati e unità di misura** (es. id utente diverso da stringa, euro diverso da dollari): previene l'errore tipico del codice generato di mescolare valori che non devono combinarsi.
- **Generici limitati**, monomorfizzati a compile time (niente costo runtime, niente dispatch dinamico di default).
- **Effetti nel tipo**: una funzione che usa file o rete lo dichiara nella firma (vedi 4.7).

4.5 Semantica

- Valutazione **stretta** (non lazy), ordine di valutazione definito.
- **Immutabilità per default**; mutazione solo esplicita e locale.

- **Errori come valori**: ogni operazione fallibile restituisce Result; un operatore di propagazione compatto sostituisce le eccezioni.
- **Nessun undefined behavior**: overflow interi definito (controllato o wrapping, da scegliere e dichiarare), accessi a indici controllati, niente aritmetica di puntatori.
- **Controllo di flusso minimo**: condizionale, pattern matching esaustivo, iterazione su collezioni e un ciclo generico. Nessun fall-through, nessun goto, nessun break implicito.
- **Un solo modo di fare ogni cosa**: dove altri linguaggi hanno tre forme di ciclo, qui ce n'è una o due.

4.6 Gestione della memoria

In Tlön. Tlön è dove vivono i programmi Aleph: il runtime che alloca la memoria, offre la libreria standard e fa da sandbox. Un programma esiste e agisce solo dentro i confini che Tlön gli concede.

- **Nessun garbage collector** nel percorso normale: velocità prevedibile, runtime minimo.
- **Regioni / arena** come modello base: la memoria si alloca in regioni legate allo scope e si libera in blocco all'uscita. Semplice per un modello da scrivere, niente borrow checker, niente leak nei casi comuni.
- **Valori per copia** per i tipi piccoli; riferimenti non nulli e validi per la durata della regione.
- **Escape**: un valore che deve sopravvivere alla regione si dichiara esplicitamente (es. promozione a regione genitore o copia). Il compilatore rifiuta i riferimenti pendenti.
- **Alternativa da valutare**: ownership stile Rust. Più potente ma i modelli scrivono male il borrow checker: è il caso tipico da misurare, non da assumere.

4.7 Effetti e capacità (sicurezza)

In Tlön. I permessi sono i confini del mondo: Tlön li impone a runtime, così un programma non può toccare nulla che non abbia dichiarato.

Ogni modulo dichiara in testa ciò di cui ha bisogno; il default è **niente**. Il compilatore verifica che nessuna funzione usi capacità non dichiarate; il runtime le impone (sandbox WASM/WASI con capacità esplicite).

```
use fs.read "/data"
use net.host "api.example.com"
use time
```

- Le capacità sono **granulari** (percorso, host, operazione) e **ispezionabili**: la vista umana elenca sempre "questo programma può fare X, Y, Z".
- Un modulo che importa un altro non eredita capacità superiori a quelle che gli sono concesse.
- È la risposta al rischio principale del codice che nessuno legge: sapere sempre cosa può fare, anche senza leggere come.

4.8 Contratti, esempi e test inline

- **Esempi verificati**: ogni funzione può (e in modalità strict deve) portare esempi input/output che il compilatore esegue in fase di check.
- **Pre/post-condizioni** e proprietà verificate con test generativi (property-based).
- **Beneficio per l'agente**: il ciclo scrivi-controlla restituisce subito un fallimento locale e preciso.
- **Beneficio per la spec**: gli esempi sono anche documentazione compatta per chi legge.

4.9 Moduli, pacchetti e FFI

In Uqbar. Uqbar è il registro: ogni modulo e ogni funzione verificata viene conservata con il suo hash e ritrovata per firma o intento. Come nel racconto, Uqbar non si sfoglia: lo trovi se sai cosa cercare.

- **Moduli indirizzati per contenuto** (hash dell'AST canonico): riproducibilità per costruzione, niente lockfile, deduplica, verifica d'integrità gratis.
- **FFI verso C** con firma dichiarata e capacità esplicita: l'accesso a librerie esistenti compensa l'ecosistema piccolo.
- **Libreria standard** piccola ma completa: stringhe, liste, mappe, JSON, CSV, regex, HTTP client/server base, file system, tempo, hashing e crittografia di base, processi. Meglio un built-in in più che un'operazione reinventata male dall'agente ogni volta.

4.10 Annotazioni di intento

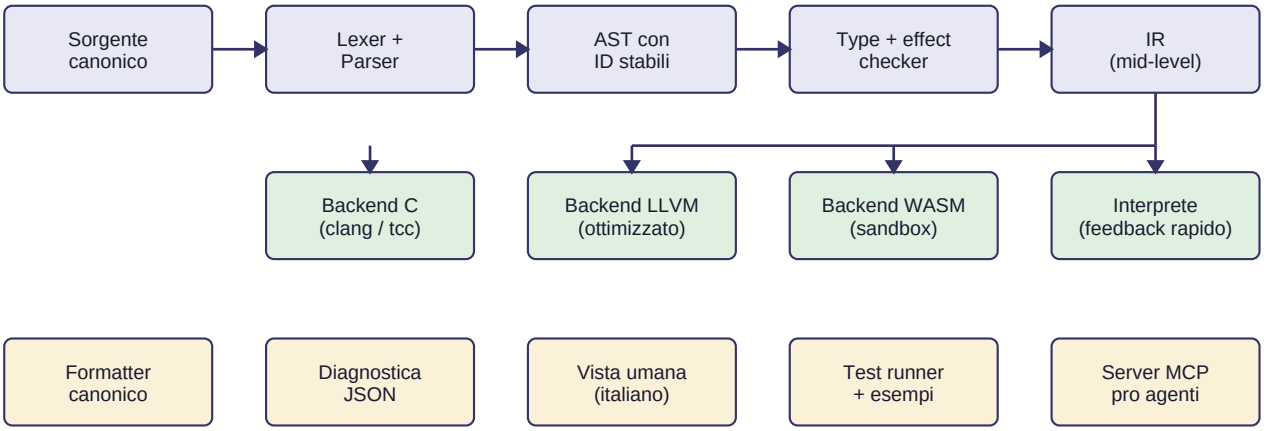
Ogni funzione può avere un'annotazione strutturata di intento (cosa deve fare, non come). Non influenza l'esecuzione. Serve alla vista umana, alle verifiche incrociate (un secondo agente controlla che il codice faccia quanto dichiarato) e alla tracciabilità (quale richiesta dell'utente ha generato quale codice).

4.11 Concorrenza (v0.3)

Fuori dalla v0.1. Direzione: concorrenza strutturata (task con scope e cancellazione) ed effetti di I/O asincrono dichiarati nei tipi.

5. Architettura del toolchain

Il compilatore è pensato come servizio per agenti, non come strumento da terminale. Pipeline e servizi:



Servizi trasversali, tutti operano sull'AST (non sul testo): chiunque può chiamarli dal server MCP.

5.1 Famiglia di componenti

I componenti prendono i nomi dal racconto di Borges *Tlön, Uqbar, Orbis Tertius*, in cui una società segreta scrive l'enciclopedia di un mondo inventato e quel mondo finisce per entrare nella realtà.

Nome	Componente	Perché
Aleph	Il linguaggio e il compilatore (comando `aleph`): spec, parser, checker, backend, server MCP.	Il punto che contiene tutto: un'unica forma per ogni programma.
Tlön	Runtime ed esecuzione: arene, stdlib nativa, sandbox WASM con capacità e budget di risorse.	Il mondo in cui i programmi vivono e agiscono.

Nome	Componente	Perché
Uqbar	Registro dei moduli e delle funzioni verificate, indirizzati per hash; ricerca per firma e intento.	La voce che esiste solo in certe copie dell'enciclopedia: la trovi se sai dove cercare.
Orbis	Pipeline del corpus: agenti che generano, verificano e pubblicano programmi; dataset, fine-tuning e benchmark.	Orbis Tertius è la società che scrive l'enciclopedia: qui sono gli agenti che scrivono il corpus.

Nota: Tlon è anche il nome dell'azienda dietro Urbit. Come nome di componente il rischio è basso, ma nel codice si usa la grafia ASCII `tlon`.

5.2 Scelte di implementazione

Componente	Scelta proposta	Alternative / note
Linguaggio del compilatore	**Rust**: enum e pattern matching adatti ai compilatori, ecosistema (clap, serde, insta, proptest) e distribuzione di un singolo binario.	OCaml (molto adatto, community più piccola), Zig. Obiettivo a lungo termine: self-hosting come prova di maturità.
Parser	Discesa ricorsiva scritta a mano (grammatica LL(1)); errori con posizione e token attesi.	Una grammatica tree-sitter in parallelo per gli editor.
AST	Albero con **ID di nodo stabili**, serializzabile in formato binario compatto e in JSON.	Gli ID permettono modifiche per nodo e diff semantici.
Type + effect checker	Inferenza locale, controllo di esaustività dei match, controllo di capacità ed effetti.	Output in diagnostica strutturata.
IR	IR di medio livello proprio, tipato, a blocchi base, con regioni esplicite.	Facilita il passaggio ai backend e le ottimizzazioni specifiche (arene).
Backend 1: C	Generazione di C99 compilato con clang o tcc. Ottimo per partire: portabile, debuggabile, tempi di compilazione rapidi con tcc.	Su tcc si ottiene un ciclo quasi istantaneo; su clang -O2 il codice finale.
Backend 2: LLVM	LLVM per ottimizzazione e target multipli.	Cranelift come alternativa a compile time ridotto.
Backend 3: WASM	Target WebAssembly eseguito in runtime come wasmtime con capacità WASI.	È la sandbox per il codice non fidato.
Interprete	Interprete per l'IR per feedback immediato e per test.	Tenuto allineato al compilatore con test differenziali.
Test	Test differenziali: stesso programma su interprete, C, LLVM e WASM devono dare lo stesso risultato.	Fuzzing della grammatica (programmi casuali validi).

5.3 Struttura del repository

```
aleph/
  spec/            specifica formale (EBNF, regole di tipo, semantica)
  lexicon/         lexicon.toml + script di ottimizzazione sui tokenizer
  compiler/
    lexer/ parser/ ast/ check/ ir/ backend-c/ backend-llvm/ backend-wasm/
    interp/        interprete di riferimento
    fmt/           stampa canonica (AST -> testo)
    view/          vista umana (AST -> italiano/inglese)
  tlon/            runtime: arene, stdlib in C/Rust, sandbox WASI
  stdlib/          libreria standard scritta nel linguaggio
  mcp/             server MCP per agenti
  bench/           suite di task, harness, risultati
  uqbar/           registro di moduli e funzioni verificate (hash)
  orbis/           pipeline del corpus, dataset, fine-tuning
  docs/            llms.txt, skill, tutorial, sito (scritto nel linguaggio)
```

5.4 Diagnostica strutturata

Ogni errore è un oggetto, non una frase. L'agente non deve interpretare prosa: riceve cosa non va, dove, cosa era atteso e una correzione proposta.

```
{ "code": "T012", "severity": "error",
  "node": "n_4821", "span": [3, 14],
  "msg": "type mismatch",
  "expected": "i64", "found": "str",
  "fix": { "op": "replace", "node": "n_4821", "with": "parse n_4821" } }
```

5.5 Server MCP per agenti

Il compilatore si espone come server MCP, così qualsiasi modello lo usa senza installare nulla. Strumenti proposti:

Strumento	Funzione
spec	Restituisce la spec compatta (con livello di dettaglio parametrico) per metterla nel contesto.
check	Controllo di sintassi, tipi, capacità ed esempi; restituisce diagnostica JSON.
build / run	Compila verso un target e/o esegue in sandbox con capacità concesse; restituisce output e metriche.
test	Esegue esempi e proprietà; restituisce falliti con input minimizzato.
edit	Applica una modifica per nodo (sostituisci, inserisci, elimina) invece di riscrivere il file.
fmt	Porta qualsiasi sorgente alla forma canonica (utile per correggere gli errori di forma dei modelli).
explain	Vista umana: cosa fa il programma, quali capacità usa, cosa dichiara l'intento.
tokens	Conta i token di un sorgente per i tokenizer configurati (strumento di misura per il progetto).

5.6 Esecuzione e deploy

In Tlön. Tlön è dove vivono i programmi Aleph: il runtime che alloca la memoria, offre la libreria standard e fa da sandbox. Un programma esiste e agisce solo dentro i confini che Tlön gli concede.

- **Sviluppo**: interprete + diagnostica + test inline, per il ciclo rapido.
- **Produzione**: binario nativo statico (C/LLVM) per prestazioni, oppure modulo WASM per sandbox.
- **Agenti a runtime**: lo scenario chiave è l'agente che scrive codice al volo, lo esegue in WASM con capacità minime e ottiene il risultato in millisecondi.

6. Metodologia sperimentale

Ogni decisione di design (prefissa o delimitata, normalizzazione dei nomi, arena o ownership, lessico) si prende con esperimenti, non con opinioni.

6.1 Suite di benchmark

15-20 task di difficoltà crescente: manipolazione di stringhe, parsing e generazione di JSON/CSV, chiamate HTTP, strutture dati, algoritmi, I/O su file, piccole API, trasformazioni di dati. Ciascuno con test automatici nascosti all'agente. Gli stessi task sono scritti di riferimento in Python, TypeScript, Rust e C.

6.2 Metriche

Metrica	Definizione
Token di output	Token generati dal modello per risolvere il task (per tokenizer).
Token di input	Spec + prompt + diagnostica rimandata al modello nei retry.
Costo totale	Token di input + output ponderati per prezzo, includendo i retry.

Metrica	Definizione
Pass@1	% di task risolti al primo tentativo senza correzioni.
Retry	Numero medio di cicli di correzione fino al successo.
Tempo di compilazione	Dal sorgente al binario (target: sotto un secondo sul caso tipico).
Prestazioni	Tempo e memoria dell'eseguibile contro C e Python.
Costo della spec	Token necessari a spiegare il linguaggio nel contesto.

6.3 Disegno degli esperimenti

- **Modelli**: almeno tre livelli di capacità della stessa famiglia (es. Haiku, Sonnet, Opus) più due-tre modelli aperti, per non ottimizzare per uno solo.
- **Ripetizioni**: ogni task più volte per modello (temperature standard), si riportano media e varianza.
- **Condizioni a confronto**: Python come base; il linguaggio con spec nel contesto; con spec + esempi; dopo fine-tuning.
- **A/B di design**: prefissa vs delimitata, nomi liberi vs normalizzati, arena vs ownership, lessico A vs lessico B.
- **Pubblicazione**: codice dell'harness, task e risultati aperti: la credibilità del progetto passa dalla riproducibilità.

6.4 Criteri di arresto o cambio direzione

Se dopo la fase 1 il linguaggio non batte Python di un margine significativo sui token *a parità di correttezza*, si ferma o si cambia approccio (per esempio limitarsi a un formato dati/DSL compatto o contribuire a un progetto esistente). Il margine minimo si concorda prima dei risultati, per non adattare l'asticella ai numeri.

7. Piano di implementazione

Milestone	Contenuto	Criterio di uscita	Stima
M0 Analisi	Benchmark dei linguaggi esistenti, harness di misura, tokenizer configurati.	Tabella comparativa su 5 task; decisione costruire / contribuire.	1-2 sett.
M1 Core	Spec v0.1 (una pagina + EBNF), lessico v0, parser, AST, stampa canonica, checker base, interprete.	Gli esempi dei 5 task girano nell'interprete; round-trip testo-AST-testo perfetto.	4-6 sett.
M2 Nativo	IR, backend C, arene, stdlib minima.	5 task compilati a nativo; benchmark di velocità contro C e Python.	4-6 sett.
M3 Agenti	Diagnostica JSON, edit per nodo, server MCP, llms.txt e skill, esempi inline.	Un agente scrive i 15-20 task usando solo MCP e spec; pass@1 misurato.	6-8 sett.
M4 Sicurezza	Capacità ed effetti, backend WASM, sandbox, vista umana.	Un programma non può fare nulla che non dichiari; la vista umana elenca le capacità.	4-6 sett.
M5 Ottimizzazione	Ottimizzazione del lessico sui tokenizer, A/B di sintassi, backend LLVM.	Riduzione di token documentata e riproducibile; scelta finale di sintassi.	4-6 sett.
M6 Corpus	Pipeline di generazione, verifica, deduplica, licenza; rilascio del corpus.	Almeno alcune migliaia di programmi verificati pubblici.	6-8 sett.

Milestone	Contenuto	Criterio di uscita	Stima
M7 Fine-tuning	Fine-tuning di un modello aperto sul corpus, valutazione sulla suite.	Il modello fine-tunato batte il baseline in-context sul benchmark.	4-6 sett.
M8 1.0	Stabilizzazione spec, edizioni, documentazione, sito, rilascio pubblico.	Spec congelata con edizione e migrazione automatica.	4 sett.

Le stime sono di partenza per un team piccolo (due persone, con agenti di supporto) e vanno riviste dopo M0. Ruoli suggeriti: uno su frontend e design del linguaggio, uno su backend, runtime e toolchain; benchmark e corpus condivisi.

8. Strategia di adozione

Problema centrale: nessun modello conosce il linguaggio, quindi all'inizio ogni agente deve impararlo dal contesto. L'adozione si costruisce su tre piani, dal più immediato al più lento, più un piano di prodotto che dia ragioni concrete per usarlo.

8.1 Piano A — in-context (da subito)

- **Spec compatta** a più livelli di dettaglio (es. 500 / 2.000 / 8.000 token), richiamabile dallo strumento spec del server MCP.
- **llms.txt** pubblico e una **skill** per i principali agenti di coding.
- **Esempi canonici** per ogni costrutto, scelti per coprire i casi dei benchmark.
- **Obiettivo misurabile**: portare il costo della spec sotto la soglia oltre cui il risparmio sui token ripaga l'investimento in contesto.

8.2 Piano B — fine-tuning (prova di fattibilità)

In Orbis. Orbis è la società di agenti che scrive l'enciclopedia di Aleph: genera programmi, li verifica e li pubblica. È da qui che il linguaggio entra nel mondo.

1. **Generazione**: agenti (anche modelli piccoli e economici) scrivono programmi per un elenco ampio di task e specifiche.
2. **Verifica**: si tengono solo programmi che passano check, esempi e test; i falliti diventano esempi di correzione con la diagnostica (dati per il ciclo di fix).
3. **Deduplica e diversità**: eliminare quasi-duplicati; coprire domini diversi; bilanciare difficoltà.
4. **Licenza e provenienza**: licenza aperta, metadati su modello generatore e verifica.
5. **Fine-tuning** (es. LoRA) di un modello aperto di coding di piccola/media taglia sul corpus.
6. **Valutazione**: stessa suite, confronto con il baseline in-context e con Python. È il risultato da mostrare: "un modello piccolo scrive questo linguaggio meglio e con meno token".

8.3 Piano C — presenza pubblica e dati di training futuri

I contenuti pubblici e di qualità sono ciò che può arrivare nei dati di training futuri dei modelli. Non abbiamo visibilità sulle pipeline dei laboratori e i tempi tra un training e l'altro sono lunghi: l'orizzonte è di anni e non va usato come unica strategia.

- **Repository aperto** con licenza permissiva per codice e per la spec; molti programmi reali e utili, con README e spiegazioni.
- **Sito e documentazione scritti nel linguaggio** (dogfooding): la vetrina è essa stessa un esempio.
- **Articoli tecnici e paper** con la metodologia e i numeri (arXiv, blog, Hacker News, community di linguaggi); post in italiano per la community locale.

- **Eval aperta** proposta alla comunità e ai laboratori, con leaderboard dei modelli.

8.4 L'idea di "riempire internet"

L'intuizione è corretta nella direzione (l'adozione dipende da quanto codice reale esiste) ma conta il come:

Da evitare	Da fare
Migliaia di siti fotocopia o pagine generate in massa con frammenti del linguaggio: i dataset deduplicano e filtrano, i contenuti ripetitivi sono trattati come spam, c'è rischio di reputazione per Qursor.	Un numero limitato di progetti pubblici **veri e utili** (tool, utility, siti reali, esempi) con repo, README, spiegazioni e licenza aperta.
Agenti che **leggono** codice tutto il giorno: non genera dati di training, l'inferenza non addestra il modello.	Agenti che **scrivono e verificano** programmi, pubblicati come corpus aperto con test e provenienza.
Pagine di contenuto senza utilità per un lettore.	Contenuti con valore indipendente: tutorial, confronti con numeri, porting di esempi noti, risposte a domande reali.

Uso consigliato dei siti: un sito vetrina del linguaggio scritto nel linguaggio; alcuni siti di clienti Qursor (con consenso e dove ha senso tecnico) come casi d'uso reali; una libreria di esempi pubblici. Qualità e utilità prima della quantità.

8.5 Piano D — prodotto e nicchie

Un linguaggio nuovo si adotta se risolve un problema concreto meglio di prima. Nicchie candidate, in ordine:

Nicchia	Perché funziona	Prova
Script al volo degli agenti in sandbox	Velocità di avvio, sicurezza con capacità, token ridotti.	Un agente che esegue centinaia di micro-programmi al minuto con permessi minimi.
Glue code e pipeline di dati	Task ripetitivi, ben testabili, tanti esempi generabili.	Pipeline ETL scritte dagli agenti Qursor per i clienti.
Backend e siti piccoli	Binario unico, nessuna dipendenza, deploy semplice.	Siti Qursor realizzati con il linguaggio come caso reale.
Edge e funzioni serverless (WASM)	Moduli piccoli, avvio rapido, isolamento.	Confronto di avvio e costo con funzioni in altri linguaggi.

8.6 Governance, licenza e identità

- **Licenza**: permissiva (Apache-2.0 o MIT) per compilatore e stdlib; spec in licenza aperta (es. CC-BY).
- **Nome e identità**: **Aleph 4 Agents** (Aleph nel testo e nel codice). Esistono altri progetti chiamati Aleph, tutti storici o di altri ambiti: un linguaggio del CWI degli anni '70, un linguaggio funzionale oggi rinominato AFNIX, un sistema di elaborazione testi di Berkeley, un sistema di inductive logic programming. Il nome completo serve a distinguersi nelle ricerche. Da fare: registrare dominio, repo e handle con il nome completo; non usare l'estensione .ale (già usata dal vecchio ALEPH).
- **Processo**: proposte di modifica alla spec pubbliche con numerazione (stile RFC) e un benchmark obbligatorio per ogni modifica sintattica.
- **Edizioni**: versioni datate del lessico e della grammatica con migrazione automatica (grazie all'AST).

8.7 Timeline di adozione

Periodo	Obiettivo	Segnali di successo
Mesi 0-3	M0-M2: analisi, core, nativo. Repo privato.	Prototipo che batte Python sui token a parità di correttezza.

Periodo	Obiettivo	Segnali di successo
Mesi 3-7	M3-M5: agenti, sicurezza, ottimizzazione. Prima spec pubblica, llms.txt, benchmark.	Un agente completa la suite con pass@1 stabile; primo articolo tecnico.
Mesi 7-10	M6-M7: corpus e fine-tuning. Repo pubblico e leaderboard.	Modello fine-tunato batte il baseline; prime stelle/contributi esterni.
Mesi 10-12	M8 1.0. Siti e progetti reali, primi casi d'uso con clienti.	Primi progetti di terzi; uso in produzione in Qursor su una nicchia.
Mesi 12-24	Estensione: concorrenza, stdlib, ecosistema. Proposta di eval ai laboratori.	Utenti non legati al team; programmi scritti da agenti ogni settimana.
Oltre 24 mesi	Possibile ingresso nei dati di training dei modelli futuri.	Modelli che scrivono il linguaggio bene senza spec nel contesto.

Metriche di adozione: programmi scritti da agenti per settimana, % di pass@1 dei modelli senza spec, numero di progetti esterni, contributi, citazioni; le stelle su GitHub da sole non bastano.

9. Altre idee

- **Transpilazione inversa**: convertire Python/JS esistenti nel linguaggio per generare corpus da codice reale e agevolare le migrazioni.
- **Compilatore come servizio** (MCP remoto): l'agente invia sorgente e riceve binario o diagnostica, senza installare nulla.
- **Profili di lessico per modello** (solo in-context): ristampa dell'AST con il lessico ottimale per un dato tokenizer.
- **Pannello di costo**: dato un task, mostra token e costo in vari linguaggi; strumento di marketing e di misura.
- **Verifica incrociata**: un secondo agente confronta l'intento dichiarato con il codice e le capacità usate.
- **Vista vocale**: spiegazione a voce di cosa fa un programma, coerente con la visione dell'interazione a voce.
- **Pacchetti con provenienza**: ogni modulo registra modello generatore, richiesta e verifiche; utile per audit.
- **Formato dati compatto** collegato (stile TOON/JSON compatto) per i dati scambiati tra agenti, che pesano in token quanto il codice.
- **Registro di funzioni verificate (Uqbar)**: Uqbar è il registro dove ogni funzione verificata resta per sempre, identificata dal suo hash. Grazie all'hash dell'AST, prima di scrivere una funzione l'agente la cerca in Uqbar per firma e intento. Il codice più economico è quello che non si genera.
- **Ricerca per firma in Uqbar** (stile Hoogle) esposta via MCP: "dammi una funzione da str a Result[Json]".
- **Budget di risorse dichiarato**: oltre ai permessi, tempo massimo, memoria e chiamate di rete, imposti dalla sandbox.
- **Trace di esecuzione strutturata**: in caso di errore, il runtime restituisce all'agente la sequenza di nodi eseguiti e i valori, non uno stack trace in prosa.
- **Patch semantiche tra agenti**: più agenti lavorano sullo stesso programma scambiandosi modifiche per nodo, senza conflitti di testo.

- **Subset dentro altri linguaggi**: una forma ridotta e canonica trasformabile in Python/TS come ponte di adozione.

10. Rischi e mitigazioni

Rischio	Mitigazione
I modelli diventano così efficienti che il risparmio di token non conta.	Il valore resta in velocità, sicurezza e correttezza; benchmark continui; criteri di arresto della sezione 6.4.
I modelli non scrivono bene il linguaggio senza training.	Spec compatta, forma canonica, diagnostica con fix proposti, grammatica vincolata, fine-tuning.
Il linguaggio non entra mai nei dati di training dei laboratori.	Piani A, B e D non dipendono da questo; il fine-tuning aperto dimostra il valore.
Codice non leggibile e non verificabile.	Esempi e test obbligatori, capacità esplicite, vista umana, verifica incrociata dell'intento.
Ambizione "universale" prematura.	Partire da una nicchia dimostrata; il linguaggio universale è un esito, non un punto di partenza.
Scarsa manutenzione (come molti progetti individuali).	Responsabile chiaro, roadmap, governance aperta, test differenziali per contenere la complessità.
Scelte di design instabili a causa dei tokenizer che cambiano.	Lessico come dato versionato, edizioni con migrazione automatica.
Reputazione: percezione di spam o hype.	Contenuti pubblici di valore reale, numeri riproducibili, nessuna generazione di massa di pagine vuote.

11. Decisioni aperte da discutere

1. **Sintassi**: prefissa a arità fissa oppure delimitata? Si decide con A/B (milestone M5), ma serve un default per partire.
2. **Nomi locali**: liberi o normalizzati? Rischio opacità contro risparmio di token.
3. **Memoria**: solo arene in v1, oppure subito un sottoinsieme di ownership?
4. **Overflow intero**: controllato (errore) o wrapping definito?
5. **Linguaggio del compilatore**: Rust, OCaml o Zig?
6. **Backend iniziale**: solo C (più rapido) o C + LLVM fin da subito?
7. **Identità**: dominio e handle per Aleph 4 Agents; estensione dei file (proposta: .aph).
8. **Licenza** definitiva e struttura (azienda, progetto aperto, fondazione).
9. **Nicchia di lancio**: sandbox per agenti o glue code per i clienti Qursor?
10. **Budget di calcolo**: quanto spendere in generazione del corpus e fine-tuning (e su quali modelli).

12. Riferimenti

Progetti e fonti citati (stato consultato il 10 novembre 2026):

- ilo — docs.rs/crate/ilo (manifesto dei principi)
- NanoLang — simonwillison.net/2026/Jan/19/nanolang/ (repo: github.com/jordanhubbard/nanolang)
- NERD — www.nerd-lang.org

- Ax — github.com/axlanguage/axlang
- NXL — github.com/sir-ad/NXL
- ll-lang — dev.to/neftedollar/why-we-built-ll-lang-a-statically-typed-functional-language-for-llms-2hg8
- KARN — dev.to/gautamkishore/karn-language-agents-speak-2a0h
- Unison — www.unison-lang.org
- Tacit — github.com/ImJasonH/ImJasonH/blob/main/articles/llm-programming-language.md
- AI-oriented grammar (paper) — arxiv.org/abs/2404.16333
- Panoramica "AI-native languages" — langpop.com/blog/ai-native-programming-languages

Le affermazioni sui progetti esistenti sono tratte dalle loro pagine pubbliche e vanno verificate nella fase 0; tutte le stime di tempi e risultati sono ipotesi di lavoro.